

Guía de Defensa — Marco Teórico de Modelos, Semana 2

Caso N.º 1 — Señales y Sistemas — Prof. Roberto Calvo Arias Tecnologías de la Información y Comunicación Empresarial · Universidad Invenio Fabrizio Espinoza Arce · Alejandro Josué Rodríguez Zamora · Jose Pablo Monestel · Isaac Felipe Morún Moreira

Esta guía cubre el **segundo** entregable: los cinco modelos del enunciado, la justificación de la elección, y el procedimiento de ocho pasos. La guía del primer entregable —series de tiempo, estacionariedad, autocorrelación, métricas— sigue siendo válida y está en [DEFENSA-marco-teorico-concisa.md](#).

Todo lo que aparece acá con un número salió de ejecutar algo. Ninguna cifra de este documento es estimada.

Cómo usar esta guía según el tiempo que tengas

Tiempo	Qué leer
10 minutos	Las 12 cifras que sí o sí, y la respuesta de 60 segundos
30 minutos	Lo anterior + Parte II (las 8 preguntas que no podés fallar)
1 hora	Todo, saltando la Parte 0 si ya tenés claros los conceptos
2 horas	Todo, en orden, en voz alta

Las 12 cifras que sí o sí

Si te preguntan cualquier cosa y no te acordás de nada más, estas doce te sacan del apuro.

#	Cifra	Qué es
1	0,3905	F1 macro del bosque aleatorio clásico. El mejor modelo del proyecto.
2	0,3686	F1 macro de Chronos-Bolt, el modelo fundacional
3	0,3457	F1 macro de iTransformer, el modelo avanzado
4	0,3368	F1 macro del baseline aleatorio. El piso que hay que superar.
5	0,3161	F1 macro del baseline trivial, que responde siempre «Continuidad»
6	+0,0537	Cuánto le gana el bosque al azar. IC 95 % [0,0137 · 0,0929], excluye el cero
7	-0,0448	Cuánto pierde iTransformer contra el bosque. IC excluye el cero : la desventaja es real
8	96,3 min contra 0,2 min	Chronos-T5 contra Chronos-Bolt sobre el mismo bloque. Por eso se eligió Bolt
9	27 segundos	Lo que tarda en entrenarse iTransformer, contra un techo de 2 horas
10	+0,00078	El aporte de los cinco activos de apoyo. Cambia de signo entre semillas

11	+0,00071	Lo que aporta añadir columnas duplicadas que no informan nada . Casi lo mismo
----	-----------------	--

12	13 114	Velas de 4 horas del panel de trabajo
----	---------------	---------------------------------------

PARTE 0 — DESDE CERO

Si alguno de estos conceptos no lo tenés claro, se lee esto primero. Están en el orden en que hacen falta.

Concepto 1: qué es un modelo fundacional de series de tiempo

Un **modelo fundacional** —*foundation model*— es un modelo grande entrenado una vez sobre una cantidad enorme de datos variados, que después se usa en problemas para los que **no fue entrenado específicamente**.

La idea viene del lenguaje: GPT se entrenó con texto de internet y después sirve para resumir, traducir o responder preguntas sin volver a entrenarlo. Un **TSFM** (*Time Series Foundation Model*) es lo mismo pero con series de tiempo: se entrena con millones de series —consumo eléctrico, ventas, tráfico web, clima— y después se le da una serie que nunca vio y pronostica.

Por qué importa acá: nosotros tenemos 420 ejemplos de la clase minoritaria en entrenamiento. Es poco. Un modelo que ya aprendió cómo se comportan las series en general puede aprovechar eso, en lugar de tener que aprenderlo desde cero con nuestros pocos datos.

Si preguntan: «Es un modelo grande preentrenado con muchísimas series de tiempo distintas, que se usa sobre una serie nueva sin volver a entrenarlo.»

Concepto 2: qué es *zero-shot*

Zero-shot quiere decir **sin ningún entrenamiento sobre nuestros datos**. Se le da el modelo tal como viene, se le muestra el historial de Litecoin, y pronostica.

Es lo contrario de *fine-tuning*, que sería seguir entrenándolo un poco con nuestros datos.

Nosotros usamos Chronos-Bolt en zero-shot puro. No le mostramos ni un ejemplo etiquetado.

Por qué eso es una ventaja para defender: un modelo que no vio nuestros datos no puede haberlos memorizado. Cualquier

resultado que dé sobre validación es genuinamente sobre datos no vistos.

Y por qué es también una limitación honesta: no sabe nada de criptomonedas en particular. Le estamos pidiendo que generalice.

Concepto 3: qué es un Transformer y qué es la «atención»

Un **Transformer** es un tipo de red neuronal que apareció en 2017 (Vaswani et al.) para traducción automática, y que hoy está detrás de casi todo lo que se llama «inteligencia artificial».

Su pieza central es el **mecanismo de atención**. La idea, en cristiano:

Para entender cada elemento de una secuencia, el modelo mira **todos los demás a la vez** y decide cuánto le importa cada uno.

Compará con lo anterior. Una red **recurrente** (una LSTM, por ejemplo) procesa la secuencia paso a paso, como si leyera una frase palabra por palabra y tuviera que acordarse de todo lo anterior en una sola memoria. Si algo importante pasó hace 200 pasos, se puede haber perdido por el camino.

La atención no tiene ese problema: **el paso 1 y el paso 200 están a la misma distancia**.

Si preguntan: «Es un mecanismo que compara cada punto de la secuencia contra todos los demás y les asigna un peso, en vez de recorrerlos en orden.»

Concepto 4: la crítica a los Transformers en series de tiempo

Acá hay un matiz que conviene tener listo, porque es exactamente lo que un profesor pregunta.

Los Transformers dominan el lenguaje. En **series de tiempo largas**, no está tan claro.

Zeng et al. (2023) mostraron que un modelo lineal simple — DLinear— **supera a varios Transformers** en las pruebas estándar de pronóstico a largo plazo. Su argumento:

La atención es *permutation-invariant*: no distingue el orden por sí sola, hay que decírselo aparte con una «codificación posicional». Y en una serie de tiempo, **el orden es el dato**.

O sea: lo que hace poderosa a la atención en lenguaje —poder mirar todo a la vez sin importar el orden— puede ser una desventaja cuando el orden es justamente lo que importa.

Esa crítica es la que motivó las variantes. Informer e iTransformer no son Transformers de lenguaje adaptados: están diseñados para series desde el principio.

Concepto 5: qué hace distinto a iTransformer

Un Transformer normal, aplicado a series, atiende **entre instantes de tiempo**: compara la vela de las 4 con la de las 8, con la de las 12, etcétera.

iTransformer invierte eso. Atiende **entre series completas**: trata a LTC entero como un elemento, a BTC entero como otro, y compara series contra series.

De ahí la «i»: *inverted*.

Por qué lo elegimos: nuestro problema es explícitamente multivariante —seis criptomonedas—, así que una arquitectura que compara series entre sí calza con la forma de nuestros datos.

Y acá viene lo honesto, que hay que decir antes de que lo pregunten: que calce con la *forma* de los datos no significa que sirva. Lo medimos. **iTransformer es el peor de los tres modelos**, y su desventaja frente al bosque clásico es la única que resulta estadísticamente distinguible.

Concepto 6: qué es un modelo de espacio de estados (y por qué CryptoMamba no es un Transformer)

Un **modelo de espacio de estados** (*state space model*, SSM) mantiene un «estado» que se va actualizando a medida que recorre la secuencia. Es más parecido a una recurrencia que a la atención: procesa en orden y arrastra memoria.

Mamba es un SSM moderno y muy eficiente. **CryptoMamba** es una aplicación de Mamba a precios de criptomonedas.

El punto que hay que saber decir: el enunciado pide un Transformer como segundo modelo y menciona CryptoMamba entre las opciones. **CryptoMamba no es un Transformer.** Son dos familias distintas, con mecanismos de cómputo distintos: recurrencia lineal contra atención cuadrática.

No es una objeción caprichosa. Elegirlo cumpliría con la lista de opciones y **no** con el requisito literal del entregable.

Y además no se puede instalar sin CUDA en ninguna de nuestras máquinas, lo que comprobamos en un entorno desechable.

Dos razones independientes. Si una no convence, la otra sigue en pie.

Concepto 7: qué son los cuantiles y por qué Bolt los devuelve

Un pronóstico puntual dice «el precio va a ser 85». Un pronóstico por **cuantiles** dice:

«Hay un 10 % de probabilidad de que esté por debajo de 78, un 50 % por debajo de 85, y un 90 % por debajo de 93.»

Es decir: además del valor más probable, te da **cuánta incertidumbre hay**.

Chronos-Bolt devuelve 9 cuantiles de una sola pasada. TimesFM devuelve solo el pronóstico puntual.

Por qué nos importó: para decidir entre Máximo, Mínimo y Continuidad, saber cuán seguro está el modelo es información útil. Un pronóstico con mucha incertidumbre no debería producir un anuncio de giro.

Concepto 8: qué es el «contexto» de un modelo

El **contexto** es cuántas observaciones hacia atrás mira el modelo para pronosticar.

Nosotros usamos **512 velas de 4 horas**, que son unos 85 días. Se eligió así por dos motivos: que el modelo vea varios ciclos completos, y que quepa dentro del contexto nativo de los tres candidatos, para que la comparación sea entre modelos y no entre recortes distintos.

Concepto 9: qué es una semilla, y por qué la mencionamos tanto

Un modelo que **se entrena** usa números aleatorios: para inicializar sus parámetros, para barajar los datos, para elegir qué variables mira cada árbol. La **semilla** es el número que fija esa aleatoriedad, de modo que la misma semilla produzca el mismo resultado.

Cambiar la semilla es cambiar la suerte, no el modelo.

Y acá está lo importante de esta entrega:

El F1 macro de iTransformer recorre de **0,3307 a 0,3611** solo cambiando la semilla. **Un rango de 0,0304**, que es más grande que el umbral de 0,02 que usamos para decir que un modelo es mejor que otro.

Consecuencia: reportar una sola corrida de un modelo entrenado es reportar suerte. Por eso todo lo que decimos del modelo avanzado va con cinco semillas.

Concepto 10: qué es un hiperparámetro y por qué buscarlos puede engañar

Un **parámetro** lo aprende el modelo. Un **hiperparámetro** lo elegís vos antes de entrenar: cuántos árboles, qué profundidad, cuánto contexto.

Buscar hiperparámetros es probar muchas combinaciones —una **rejilla**— y quedarse con la mejor.

El problema, y es el hallazgo de esta semana: si las combinaciones se diferencian en menos de lo que se mueve el modelo al cambiar de semilla, **elegir la mejor no elige la mejor configuración, elige la semilla más afortunada.**

Lo medimos: la dispersión entre combinaciones es **0,020981**, y la dispersión entre semillas dentro de una misma combinación es **0,026073**. El ruido es más grande que la señal.

Por eso no coronamos ninguna ganadora, aunque una era numéricamente mejor. El criterio de no hacerlo estaba escrito **antes** de mirar el resultado.

Concepto 11: qué es un intervalo de confianza, en cristiano

Cuando decimos «el bosque le gana al azar por 0,0537», eso es **una** medición sobre **un** conjunto de datos. Si hubiéramos tenido otros datos, habría dado otro número.

El **intervalo de confianza del 95 %** dice: *«si repitiéramos la medición muchas veces, el 95 % de los resultados caería acá dentro»*.

La regla práctica, que es lo único que hay que recordar:

Si el intervalo incluye el cero, no podemos afirmar que haya diferencia. Aunque el número del medio sea positivo.

Ejemplo real nuestro: Chronos-Bolt le gana al baseline aleatorio por +0,0318, que suena bien. **Pero el intervalo va de -0,0030 a +0,0652 e incluye el cero.** Así que no podemos afirmar que le gane.

Concepto 12: qué es un *baseline* y por qué son obligatorios

Un **baseline** es un modelo deliberadamente tonto que sirve de piso.

Tenemos tres:

Baseline	Qué hace	F1 macro
Trivial	Responde siempre «Continuidad»	0,3161
Mayoritario	Responde la clase más frecuente	0,3161
Aleatorio	Responde al azar respetando las proporciones	0,3368

Para qué sirven: si un modelo elaborado no le gana al que responde al azar, **no aprendió nada**. Y con clases desbalanceadas —el 90 % de nuestras velas son «Continuidad»— es más fácil de lo que parece parecer bueno sin serlo.

Las palabras que tenés que poder decir sin dudar

Palabra	En una frase
TSFM	Modelo grande preentrenado con muchas series, que se usa sobre una nueva sin reentrenar
Zero-shot	Sin ningún entrenamiento sobre nuestros datos
Atención	Cada punto se compara contra todos los demás y les asigna un peso
Transformer	Red basada en atención, de 2017, nacida para lenguaje
iTransformer	Transformer que atiende entre series completas en vez de entre instantes
SSM	Modelo que arrastra un estado y procesa en orden; no es atención

Cuantil	Un nivel de probabilidad del pronóstico; dice cuánta incertidumbre hay
----------------	--

Semilla	El número que fija la aleatoriedad de un modelo que se entrena
----------------	--

Hiperparámetro	Lo que elegís antes de entrenar, no lo que el modelo aprende
-----------------------	--

Baseline	Modelo tonto que sirve de piso obligatorio
-----------------	--

PARTE I — RESUMEN COMPACTO

La respuesta de 60 segundos

Si te preguntan «¿qué hicieron esta semana?», esto:

Revisamos los cinco modelos que pide el enunciado y elegimos dos: **Chronos-Bolt** como fundacional y **iTransformer** como avanzado. Las dos elecciones salieron de medir en nuestras máquinas, no de popularidad. Y los medimos. **Chronos-Bolt saca 0,3686 de F1 macro; iTransformer, 0,3457. El bosque aleatorio clásico, que ya teníamos, saca 0,3905.** O sea que **el modelo más simple sigue siendo el mejor**, y la desventaja de iTransformer frente a él es la única que resulta estadísticamente distinguible. No es el resultado que esperábamos, pero está bien medido, y preferimos reportarlo que maquillarlo.

Lo que entregamos, en números

Modelos del enunciado revisados	5
Modelos descartados con motivo medido	3 (CryptoMamba, VTA, FinLSPM)
Candidatos a fundacional medidos en CPU	3
Modelos evaluados sobre el mismo bloque	4
Decisiones registradas con evidencia	16
Números del documento respaldados por una medición	421 de 421

PARTE II — LAS 8 PREGUNTAS QUE NO PODÉS FALLAR

1. «¿Por qué eligieron Chronos-Bolt y no Chronos-T5 o TimesFM?»

Por presupuesto de tiempo, medido, no por preferencia.

Los tres candidatos corren en CPU. Los separa el tiempo de inferencia:

Candidato	Disco (MB)	RAM pico (MB)	Bloque de validación
chronos-bolt-small	182,05	695,0	0,2 min
chronos-t5-small	176,08	4 805,6	96,3 min
timesfm-2.5-200m	882,32	1 264,6	6,0 min

Chronos-T5 tarda 96,3 minutos de sola inferencia, sin entrenar nada, contra un techo de dos horas que nos fijamos. Y usa 4,8 GB de memoria contra 695 MB.

Por qué esa diferencia: T5 es autorregresivo y muestrea 20 trayectorias por ventana. Bolt predice sus 9 cuantiles de una sola pasada. **Es 484 veces más rápido por ventana.**

TimesFM es viable (6 minutos), pero ocupa 4,8 veces más disco y devuelve solo el pronóstico puntual, sin cuantiles.

Si insisten: «El orden de magnitud es lo que decide, y ese no cambia con la configuración.»

2. «¿Por qué descartaron CryptoMamba, si el enunciado lo menciona?»

Dos razones independientes.

1. **No es un Transformer.** Es un modelo de espacio de estados. El enunciado pide expresamente un Transformer como segundo modelo. Elegirlo cumpliría con la lista de opciones y no con el requisito.
2. **No se puede instalar sin CUDA** en ninguna de nuestras máquinas. Comprobado en un entorno desechable.

Lo importante de cómo lo planteamos: la primera razón es una ambigüedad del enunciado, no una decisión nuestra. La llevamos a consulta con el profesor en vez de resolverla por cuenta propia.

3. «¿Y VTA y FinLSPM?»

VTA (Verbal Technical Analysis) convierte los indicadores técnicos a texto para que un modelo de lenguaje los interprete. El costo lo descarta: habría que textificar **78 684 valores** (13 114 velas $\times$ 6 activos). No lo medimos exactamente, y lo decimos así: es un costo cualitativamente alto, no una cifra que inventamos.

FinLSPM falla un criterio distinto y más simple: el enunciado exige código público, y **no lo encontramos**.

Ojo con cómo se dice esto, porque es un punto de honestidad que vale:

No decimos «no existe». Decimos «**no lo hemos encontrado**». Son cosas distintas y solo una de las dos la podemos sostener.

Un detalle que suma si sale: el argumento inicial para descartar FinLSPM era que está pensado para acciones, que tienen anclaje fundamental, y las criptomonedas no. **Fuimos al artículo y descubrimos que también lo evalúan sobre Bitcoin**, así que ese argumento no se sostenía y lo reemplazamos por el de disponibilidad de código, que sí es verificable.

4. «¿Por qué iTransformer y no Informer?»

Porque Informer no lo pudimos instalar, y eso no es lo mismo que decir que no sirve.

Ruta intentada	Qué trae	Resultado
iTransformer	iTransformer	resoluble
neuralforecast	Informer e iTransformer	no resoluble: depende de ray, sin distribuciones para Python 3.14 en Windows
informer-pytorch	Informer	no resoluble

neuralforecast era la ruta obvia porque traía los dos juntos.

Cómo se reporta, que es lo que importa: no es una prueba de que Informer sea inusable en general. **Es lo que pudimos verificar en nuestro entorno**, y así está escrito.

Si preguntan por qué no copiaron el código a mano: meter código ajeno sin empaquetar en el repositorio no se justifica cuando el otro candidato del enunciado sí está disponible.

5. «El modelo avanzado es peor que el bosque. ¿Entonces para qué lo hicieron?»

Esta es la pregunta incómoda, y la respuesta es buena.

Primero los números, sin esconder ninguno:

Modelo	F1 macro	Contra el bosque
Bosque aleatorio clásico	0,3905	—
Chronos-Bolt (fundacional)	0,3686	−0,0219, IC incluye el cero
iTransformer (avanzado)	0,3457	−0,0448, IC excluye el cero

iTransformer es el único cuya desventaja es distinguible del ruido. Los otros dos no se distinguen entre sí.

Y la respuesta:

El enunciado pide comparar un modelo fundacional contra uno avanzado. **Lo hicimos y el resultado es que ninguno de los dos mejora al modelo clásico.** Eso es un resultado, no un fracaso. Lo que no podíamos hacer era elegir sin medir. Si hubiéramos supuesto que un Transformer sería mejor porque es lo moderno, habríamos escrito una conclusión sin respaldo.

El remate, si querés cerrarlo fuerte:

Con 420 ejemplos de la clase minoritaria, un modelo con atención tiene mucho que ajustar y poco de dónde. Que un bosque aleatorio le gane es coherente con lo que ya sabíamos de nuestros datos, no una sorpresa.

6. «Dijeron que el enfoque multivariante era el planteamiento del caso. ¿Aportan o no aportan los otros cinco activos?»

Medido: no se puede afirmar que aporten. Y conviene decirlo con el control al lado, porque es lo que lo hace concreto.

Comparación	Media	¿Signo estable?
Bosque contra el baseline aleatorio	+0,04065	sí, mínimo +0,0296
Aporte de los cinco activos de apoyo	+0,00078	no, 2 de 5 semillas negativas
<i>Columnas duplicadas que no informan nada</i>	+0,00071	no

Las dos últimas filas son el punto. La tercera es un control: le añadimos al modelo ocho columnas que reemiten información que ya estaba, con otro nombre. Por construcción no aportan nada.

Lo que se mide al incorporar los cinco criptoactivos es del mismo tamaño que lo que se mide al no incorporar nada.

La distancia entre ambos es menor que la trescientava parte del umbral de decisión.

Y lo confirmamos desde otra familia de modelo.

iTransformer es la arquitectura cuyo argumento de venta es atender *entre series*: si aportaran, ahí tendría que verse. Da **+0,0126** de media sobre cinco semillas, por debajo del umbral de 0,02.

Lo que NO hay que decir: que el planteamiento estaba mal. La justificación de diseño se sostiene —las correlaciones medidas en la Semana 1 la respaldan—. **Lo que cambia es la conclusión, no el método.**

7. «¿Optimizaron los hiperparámetros?»

Sí, y el resultado es que no sirvió de forma distinguible. Las dos mitades tienen su propia historia.

El fundacional (determinista): probamos 18 combinaciones de tamaño de modelo, contexto y cuantil. La mejor da +0,018680 sobre la configuración por defecto, **con un intervalo que incluye el cero**. O sea que la ganancia no se distingue del efecto de haber elegido el máximo de una rejilla mirando el mismo bloque.

El avanzado (se entrena): acá está el detalle bueno.

La dispersión **entre combinaciones** es 0,020981. La dispersión **entre semillas dentro de una misma combinación** es 0,026073. **El ruido es más grande que la señal.**

Y no coronamos ganadora, aunque una combinación era numéricamente mejor que la por defecto (0,346696 contra 0,345129).

Por qué eso es defendible y lo contrario no: el criterio de no coronar estaba escrito **antes** de correr la rejilla. Si hubiéramos elegido esa combinación y la reportáramos contra el bloque de prueba, habríamos publicado la celda con las semillas más afortunadas, y nadie lo habría podido detectar desde fuera.

8. «¿Cómo pasan de un pronóstico de precios a una etiqueta de tres clases?»

Buena pregunta, y hay que tener lista la respuesta porque es una decisión de diseño real.

Un modelo fundacional **pronostica el precio**, no la clase. Había dos caminos:

1. Usar el modelo congelado como extractor de representaciones y entrenar encima una cabeza de clasificación.
2. **Pronosticar la trayectoria y aplicarle la misma función de etiquetado del contrato**, la que usamos con los datos reales.

Elegimos la segunda, y lo decidió una medición: etiquetar la trayectoria cuesta unos **12 segundos** sobre el bloque de validación entero. La opción simple resultó ser también la barata, así que la otra habría tenido que justificar un costo adicional contra algo que prácticamente no cuesta.

Y la limitación que trae, que declaramos nosotros antes de que la pregunten:

El etiquetador exige que los **14 vecinos** sean *estrictamente* menores que el centro para marcar un máximo. Sobre precios reales los empates son improbables. Sobre una trayectoria **pronosticada**, que tiende a ser suave, diferencias diminutas deciden la etiqueta: comprobamos que un vecino situado 10^{-11} por debajo del centro cambia el resultado de Continuidad a Máximo. Por eso el modelo avanzado no da el mismo número en dos corridas con la misma semilla: difieren en 0,004833.

No es un defecto del etiquetador. Es el precio de esta vía del puente, y afecta a cualquier modelo que pronostique una trayectoria suave.

PARTE III — LAS PREGUNTAS INCÓMODAS

«¿No será que su implementación está mal?»

Respuesta honesta y verificable:

Puede ser, y por eso pusimos controles. **Antes de publicar cualquier número nuevo, el procedimiento tiene que reproducir un número que ya conocíamos.** Si no lo reproduce, el script se detiene y no escribe nada. Y hay un caso que lo respalda: el F1 del bosque, **0,390497720487045**, lo obtuvieron dos personas por separado, con código distinto y con propósitos distintos. **Quince decimales iguales.**

«Ustedes eligieron el umbral de 0,02. ¿No es arbitrario?»

Sí, es una convención del equipo, y está declarada como tal. Por eso la regla que usamos es: **cuando el margen y su intervalo de confianza discrepan, manda el intervalo.** El umbral orienta; el intervalo decide.

«¿Por qué no reportan resultados sobre el conjunto de prueba?»

Porque se gasta una sola vez. Todo lo que hemos reportado es sobre **validación**. El bloque de prueba no se ha tocado, y se mide al final, cuando la configuración esté fija. Medirlo ahora dejaría al informe final sin datos no vistos.

«Su mejor modelo apenas le gana al azar. ¿Sirve de algo?»

No hay que ponerse a la defensiva acá.

El bosque le gana al baseline aleatorio por **0,0537**, con un intervalo que **excluye el cero**, y la ventaja sobrevive al reentrenamiento: sobre cinco semillas la menor diferencia es 0,0296 y ninguna cae por debajo del umbral. Es una ventaja real y es pequeña. **El problema es duro:** predecir puntos de inflexión en un mercado es difícil por naturaleza, y ya habíamos medido en la

Semana 1 que la señal disponible es escasa. Un margen honesto y chico vale más que uno grande que no se sostenga.

Lo que NO hay que decir

No digas	Decí
«FinLSPM no existe»	«No encontramos su código público»
«Informer no sirve»	«No pudimos instalarlo en nuestro entorno»
«Los activos de apoyo no aportan»	«No podemos afirmar que aporten»
«El Transformer falló»	«Quedó por debajo del bosque, con una diferencia distinguible»
«Optimizamos y mejoró»	«Optimizamos y la mejora no se distingue del ruido»
«El modelo tiene 0,3457»	«Da 0,3457 en una corrida; entre semillas se mueve 0,0304»

Los tres remates que funcionan

Si te preguntan por el resultado negativo:

«Cambia la conclusión, no el método.»

Si te preguntan por qué no eligieron lo más moderno:

«Porque el enunciado pide justificar con las características medidas de los datos y de nuestras máquinas, no por popularidad.»

Si te preguntan por la rejilla que no coronó ganadora:

«El criterio estaba escrito antes de mirar el resultado. Si lo hubiéramos escrito después, se llamaría justificar lo que ya queríamos.»